



**ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ**
UNIVERSITY OF PATRAS

**ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ
ΤΕΧΝΟΛΟΓΙΑΣ ΥΠΟΛΟΓΙΣΤΩΝ**

**Μηχανική Μάθηση – Μέθοδοι επεξεργασίας με
τεχνικές Μηχανικής Μάθησης**

1^η εργασία

Ον/μο: Σάλιαρης Γεώργιος

ΑΜ: 1092584

Υπεύθυνος καθηγητής: Μουστακίδης Γεώργιος

Ακαδημ. έτος: 2024-25

Ζητούμενα της εργασίας

Πρόβλημα 1

Problem 1

①

a) $X = [X_1, X_2]$

$H_0: f_0(x_1, x_2) = f_0(x_1) \cdot f_0(x_2)$ με $f_0(x) \sim N(0, 1)$

$H_1: f_1(x_1, x_2) = f_1(x_1) f_1(x_2)$ με $f_1(x) \sim 0.5 [N(-1, 1) + N(1, 1)]$

Ισχύει επίσης ότι:

$$P(H_0) = P(H_1) = 0.5$$

Θέλουμε να βρούμε το βέλτιστο τεστ κατά Bayes το οποίο ελαχιστοποιεί την πιθανότητα σφάλματος

Ισχύει ότι:

Εάν η τ.μ. $X \sim N(\mu, \sigma^2)$, τότε

έχει συνάρτηση πυκνότητας πιθανότητας:

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad \text{①} \quad \mu: \text{ μέση τιμή της τ.μ. } X \text{ } \sigma^2: \text{ η διακύμανση της } X$$

Εμείς γνωρίζουμε από την θεωρία ότι το βέλτιστο τεστ κατά Bayes έχει τη μορφή:

$$\frac{f_1(X)}{f_0(X)} \underset{H_0}{\overset{H_1}{\geq \leq}} \frac{P(H_0)}{P(H_1)} \quad \text{②} \quad \text{με } P(H_0), P(H_1) \text{ οι εκ των προτέρων γνωστές πιθανότητες εμφάνισης των σεναρίων } H_0 \text{ κ' } H_1 \text{ αντίστοιχες}$$

② ερμηνεύεται ως εξής:

- Εάν ισχύει το ' $\geq$ ', αποφασίζουμε υπέρ της H_1 .
- Εάν ισχύει το ' $\leq$ ', αποφασίζουμε υπέρ της H_0 .
- Εάν δέ ισχύει το ' $=$ ', αποφασίζουμε αδιαφόρητα είτε υπέρ της H_0 είτε υπέρ της H_1 -δεν έχει ιδιαίτερη σημασία.

$$\frac{f_1(x_1) \cdot f_2(x_2)}{f_0(x_1) \cdot f_0(x_2)} \underset{H_0}{\overset{H_1}{\geq}} \frac{0.5}{0.5} = 1$$

$$\frac{0.5 [N(-1,1) + N(1,1)]_{x_1} \cdot 0.5 [N(-1,1) + N(1,1)]_{x_2}}{N(0,1)_{x_1} \cdot N(0,1)_{x_2}} \underset{H_0}{\overset{H_1}{\geq}} 1 \quad (1)$$

$$\frac{0.5^2 [N(-1,1) + N(1,1)]_{x_1} [N(-1,1) + N(1,1)]_{x_2}}{[N(0,1)]_{x_1} [N(0,1)]_{x_2}} \underset{H_0}{\overset{H_1}{\geq}} 1 \quad (2)$$

Οπώς $N(-1,1) = \frac{1}{\sqrt{2\pi}} e^{-\frac{(x+1)^2}{2}} \quad (4)$

$N(1,1) = \frac{1}{\sqrt{2\pi}} e^{-\frac{(x-1)^2}{2}} \quad (5)$

και $N(0,1) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2} \quad (6)$

Αρα $(3) :$

$$\frac{0.5^2 \left(\frac{1}{\sqrt{2\pi}} \right)^2 \left[e^{-\frac{(x_1+1)^2}{2}} + e^{-\frac{(x_1-1)^2}{2}} \right] \left[e^{-\frac{(x_2+1)^2}{2}} + e^{-\frac{(x_2-1)^2}{2}} \right]}{\left(\frac{1}{\sqrt{2\pi}} \right)^2 e^{-\frac{x_1^2}{2}} e^{-\frac{x_2^2}{2}}} \underset{H_0}{\overset{H_1}{\geq}} 1 \quad (7)$$

$$\frac{0.5^2 \left[e^{-\frac{(x_1+1)^2}{2}} + e^{-\frac{(x_1-1)^2}{2}} \right] \left[e^{-\frac{(x_2+1)^2}{2}} + e^{-\frac{(x_2-1)^2}{2}} \right]}{e^{-\frac{x_1^2 + x_2^2}{2}}} \underset{H_0}{\overset{H_1}{\geq}} 1 \quad (7)$$

Επομένως (7) συνιστά το ζητούμενο βέλτιστο test
κατά Bayes

b) Σε συνέχεια του προηγούμενου ερωτήματος πάλι έχουμε 1 τυχαίο διάνυσμα $X = [x_1, x_2]$ και τις εξής 2 υποθέσεις :

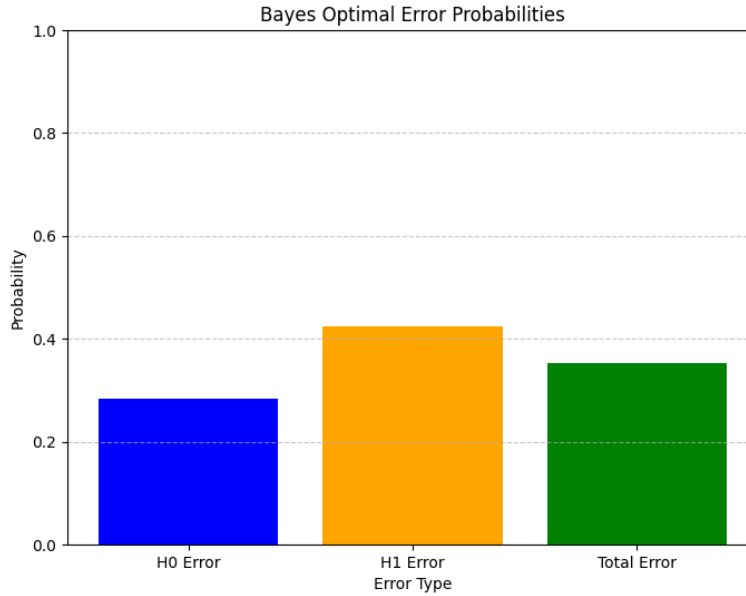
$H_0 : x_1, x_2$ ανεξάρτητα με συνάρτηση πυκνότητας πιθανότητας $f_0(x_1, x_2) = f_0(x_1) * f_0(x_2)$ με $f_0(x) \sim N(0,1)$ (1) (κανονική κατανομή μέσης τιμής 0 και διασποράς 1) και

$H_1 : x_1, x_2$ ανεξάρτητα με συνάρτηση πυκνότητας πιθανότητας $f_1(x_1, x_2) = f_1(x_1) * f_1(x_2)$ με $f_1(x) \sim 0.5 [N(-1,1) + N(1,1)]$ (2) (άρα με πιθανότητα 0.5, $f_1(x) \sim N(-1,1)$ και με πιθανότητα 0.5, $f_1(x) \sim N(1,1)$)
Ενώ για τις εκ των προτέρων γνωστές πιθανότητες των H_0 και H_1 ισχύει ότι:

$$P(H_0) = P(H_1) = 0.5 \quad (3)$$

Τώρα για θέλουμε να βρούμε την πιθανότητα σφάλματος που προκύπτει από το βέλτιστο τεστ κατά Bayes. Γι' αυτό το λόγο δημιουργούμε 10^6 δείγματα από την $f_0(x_1, x_2)$ και άλλα τόσα δείγματα από την $f_1(x_1, x_2)$. Σε αυτό το σημείο να επισημάνουμε ότι η δημιουργία των δειγμάτων για την 1^η κατανομή είναι προφανής, ενώ για τη 2^η κατανομή, κατασκευάζουμε μια άλλη τυχαία μεταβλητή η οποία ακολουθεί ομοιόμορφη κατανομή στο διάστημα $[0,1]$. Αν προκύψει λοιπόν τιμή μεγαλύτερη του 0.5, τότε θεωρούμε ότι $x_i \sim N(1,1)$ ($i = 1,2$) (4) ενώ σε διαφορετική περίπτωση $x_i \sim N(-1,1)$ ($i = 1,2$) (5). Επομένως το συνολικό σφάλμα είναι :

$$P_{error} = 0.5 * (P_{error,H_0} + P_{error,H_1}) = 0.5 * (0.2823 + 0.4246) = 0.3534 \quad (6).$$



Εικόνα 1: Εφαρμογή του βέλτιστου τεστ κατά Bayes για τον υπολογισμό του συνολικού λάθους. Αξίζει να προσέξουμε ότι τα οι πιθανότητες λάθους για τα 2 σενάρια δεν είναι οι ίδιες.

c) Εδώ ουσιαστικά ζητείται να κάνουμε train ένα νευρωνικό δίκτυο με 200 δεδομένα και εν συνεχεία να γίνει σύγκριση με το βέλτιστο τεστ κατά Bayes. Συγκεκριμένα, έχουμε 2 είδη νευρωνικών δικτύων. Και τα 2 νευρωνικά έχουν input size 2, ένα *hidden layer* με μέγεθος 20 (20 νευρώνες δηλαδή) και 1 output, ενώ και τα 2 δίκτυα εκτιμούν την *'likelihood ratio function'* :

$$r(x_1, x_2) = \frac{f_1(x_1, x_2)}{f_0(x_1, x_2)} = \frac{f_1(x_1) * f_1(x_2)}{f_0(x_1) * f_0(x_2)} \quad (7)$$

Για τη δε αρχικοποίηση των παραμέτρων και των 2 νευρωνικών δικτύων, έχουμε ότι *offset (bias) = 0* και οι αρχικές τιμές των πινάκων των βαρών για κάθε layer ακολουθούν κανονική κατανομή μέσης τιμής 0 και διασποράς $\frac{1}{n+m}$ (με άλλη αρχικοποίηση των παραπάνω παραμέτρων θα προκύψουν εν τέλει διαφορετικά αποτελέσματα μιας και τα νευρωνικά δίκτυα είναι ιδιαίτερα ευαίσθητα ως προς την αρχικοποίηση των παραμέτρων τους).

Το ένα νευρωνικό δίκτυο χρησιμοποιεί την cross-entropy method με

$$\varphi(z) = -\log(1 - z) \text{ και } \psi(z) = -\log z \quad (8)$$

Ενώ η συνάρτηση απώλειας είναι ίση με

$$Cross - Entropy loss = \frac{-1}{N} \sum_{i=1}^N (y_i \log(y_i') + (1 - y_i) \log(1 - y_i')) \quad (9)$$

όπου :

N είναι ο αριθμός των δειγμάτων (10^6)

y_i αντιστοιχεί στο πραγματικό label (0 ή 1)

y_i' στην προβλεπόμενη τιμή για ένα μεμονωμένο δείγμα (έξοδος της *sigmoid* η οποία εφαρμόζεται στην έξοδο του νευρωνικού δικτύου)

και μας δείχνει πόσο καλά οι προβλεπόμενες πιθανότητες συμβαδίζουν με τα πραγματικά *binary labels*. Προσεγγίζει την 'posterior' πιθανότητα $\frac{r(x_1, x_2)}{1 + r(x_1, x_2)}$ η οποία συγκρίνεται με το 0.5 προκειμένου να αποφασίσουμε υπέρ της H_0 είτε υπέρ της H_1 .

Το άλλο νευρωνικό δίκτυο χρησιμοποιεί την exponential method με

$$\varphi(z) = e^{0.5z} \text{ και } \psi(z) = e^{-0.5z} \quad (10)$$

με την αντίστοιχη συνάρτηση συνολικής απώλειας να είναι ίση με :

$$Exponential loss = \frac{1}{N} \sum_{i=1}^N e^{-y_i f(x_i)} \quad (11)$$

Εδώ όμως $y_i \in \{-1, +1\}$, είναι το μετασχηματισμένο label το οποίο αντιστοιχεί στο i-στο δείγμα. Ειδικότερα, εφαρμόζουμε το μετασχηματισμό:

$$y_i \rightarrow 2y_i - 1 \quad (12)$$

Ενώ $f(x_i)$ είναι το raw output / logit του μοντέλου για το i-οστό δείγμα (δεν έχουμε activation function στο layer της εξόδου).

και υπολογίζει (καλύτερα: προσεγγίζει) την ποσότητα $\log(r(x_1, x_2))$ και τη συγκρίνει με το 0 πάλι για να αποφασίσουμε υπέρ της H_0 / H_1 .

Στο σημείο αυτό να σημειωθεί ότι η σύγκλιση των 2 μεθόδων θα μπορούσε να

επαληθευτεί και μέσω της συνάρτησης $\varphi(u(x_{1,t}^0, x_{2,t}^0, \theta_t)) + \psi(u(x_{1,t}^0, x_{2,t}^0, \theta_t))$

Σε κάθε περίπτωση βέβαια, επιβεβαιώνουμε τη σύγκλιση των 2 μεθόδων βρίσκοντας τη μέσο κόστος του τρέχοντος κόστους και των 19 προηγούμενων τιμών του κάτω το οποίο γίνεται κάθε 50 *epochs* (1 *epoch* είναι ουσιαστικά ένα πλήρες πέρασμα από τα δεδομένα). Τώρα εμείς θέλουμε να ελαχιστοποιήσουμε τις συναρτήσεις απώλειας (*loss functions*) όπως ορίστηκαν παραπάνω και άρα να ενημέρωνουμε συνεχώς τις

παραμέτρους θ του εκάστοτε νευρωνικού δικτύου. Για αυτό το λόγο χρησιμοποιούμε τον αλγόριθμο *Stochastic Gradient Descent*, κανονικοποιώντας το αντίστοιχο *gradient* (ADAM optimization). Είναι δηλαδή :

$$\theta_t = \theta_{t-1} - \frac{\mu \nabla_{\theta} L}{\sqrt{c + P_{\theta}(t)}} \quad (13)$$

Όπου:

θ_t : Αντιστοιχεί στις παραμέτρους του δικτύου που μας ενδιαφέρουν

μ : *Learning rate/step size*

$\nabla_{\theta} L$: *Gradient της loss function wrt (with respect to) θ .*

c : Μια σταθερά με σχετικά μικρή τιμή ($10e-8$) που αποτρέπει τη διαίρεση με το 0.

$$P_{\theta}(t) = (1 - \lambda) * P_{\theta}(t - 1) + \lambda(\nabla_{\theta} L)^2 \quad (14),$$

Όπου στην (14):

$P_{\theta}(t)$: Προσέγγιση της ισχύος για την επανάληψη t

λ : *decay rate* (= 0.999)

$(\nabla_{\theta} L)^2$: *Element wise τετράγωνο του gradient της συνάρτησης απώλειας.*

Για τις δε παραμέτρους των νευρωνικών δικτύων και τις σχέσεις μεταξύ τους έχουμε τα εξής :

W_1 : *Pre-activation output του hidden layer* ($10^6, 20$).

X : Ο πίνακας δεδομένων με μέγεθος ($10^6, 2$)

A_1 : Ο πίνακας βαρών για το 1^ο *layer – hidden* (2,20)

B_1 : Ο πίνακας biases για το 1^ο *layer-output* (1,20)

Είναι:

$$W_1 = X * A_1 + B_1 \quad (15)$$

Και:

$$Z_1 = f_1(W_1) \quad (16)$$

Με :

f_1 : *Activation function για το hidden layer – ReLu*

Z_1 : *Activation output για το hidden layer.*

Ομοίως:

W_2 : *Pre-activation output του output layer* ($10^6, 1$).

A_2 : Ο πίνακας βαρών για το 2^ο *layer – hidden* (20,1)

B₂ : Ο πίνακας *biases* για το 1^ο *layer-output* (1,20)

Είναι:

$$W_2 = Z_1 * A_2 + B_2 (17)$$

Ενώ για την ενεργοποίηση της εξόδου έχουμε:

-*Cross-entropy method*

$$Y = f_2(W_2) = \sigma(W_2) = \frac{1}{1+e^{-W_2}} (18)$$

-*Exponential method*

$$Y = f_2(W_2) = W_2 (19)$$

Τέλος να σημειωθεί ότι για τον υπολογισμό των σχετικών παραγώγων, χρησιμοποιήθηκε το αρχείο *derivatives.pdf* . Ενδεικτικά παρακάτω παρατίθενται οι σχετικοί υπολογισμοί για την *cross-entropy method*.

$$U_2 = \nabla_Y \phi(Y) = - \left(\frac{T}{Y} - \frac{1-T}{1-Y} \right) = Y - T (20) \text{ (αφού } Y \text{ η } sigmoid) T$$

: *true labels* (0 ή 1)

– *targets, Y: προβλεπόμενα outputs του νευρωνικού δικτύου*

$$V_2 = U_2 \odot f_2'(W_2) = \dots$$

= $Y - T$ (21) – λόγω ιδιοτήτων της *sigmoid* (ο τελεστής

$\odot$ αντιστοιχεί σε *element*

– *wise* πολλαπλασιασμό των 2 εμπλεκόμενων πινάκων

$$U_1 = A_2^T V_2 = \dots (22)$$

$$V_1 = U_1 \odot f_1'(W_1) \text{ (} f_1 : ReLu \text{)} (23)$$

$$f_1'(W_1) = \{1, \text{ if } W_1 > 0 \text{ else } 0\} (24)$$

$$\nabla_{A_1 B_1} \phi(Y) = V_1 [Z_1^T \ 1] (25)$$

$$\nabla_{A_2 B_2} \phi(Y) = V_2 [Z_1^T \ 1] (26)$$

$$U_0 = A_1^T V_1 = \dots (27)$$

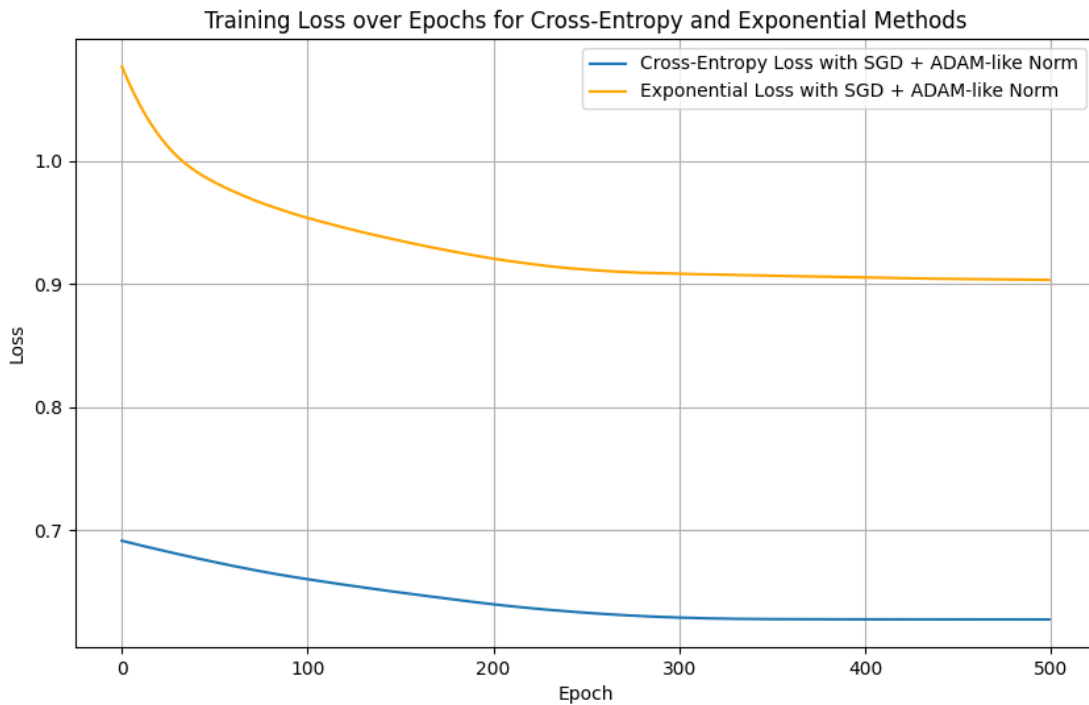
$$\nabla_Z \phi(Y) = U_0 = \dots (28)$$

Για την *exponential method* τα πράγματα είναι αρκετά παρόμοια, μόνο που:

$$\frac{\partial \text{Loss}}{\partial f(x_i)} = -y_i e^{-y_i f(x_i)} \quad (29)$$

Παρατήρηση: Οι εμπλεκόμενες παράγωγοι οι οποίες δεν υπολογίζονται ρητά υπολογίζονται από τον σχετικό κώδικα.

Τρέχοντας λοιπόν τον σχετικό κώδικα λαμβάνουμε τα εξής αποτελέσματα :

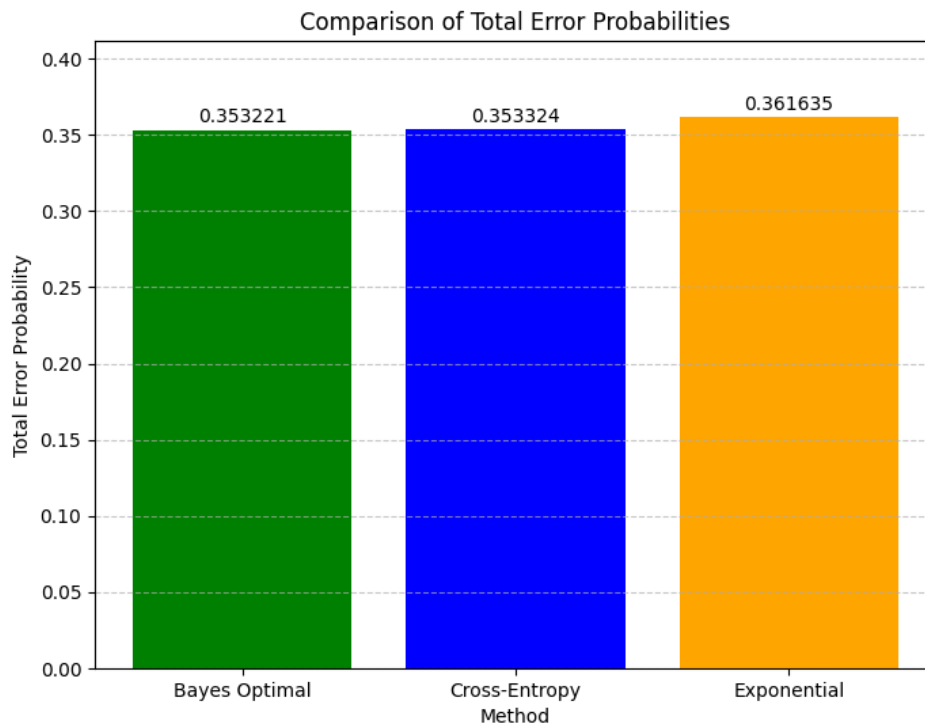


Εικόνα 2: Learning curve για το νευρωνικό δίκτυο το οποίο εφαρμόζει τη μέθοδο cross-entropy και αντίστοιχη καμπύλη για το ίδιο νευρωνικό δίκτυο το οποίο εφαρμόζει την exponential μέθοδο

Σύμφωνα με αυτήν την εικόνα παρατηρούμε σημαντική διαφορά στη σύγκλιση ανάμεσα στις 2 μεθόδους, πράγμα το οποίο ενδεχομένως να οφείλεται στη διαφορετική μορφή που έχει η *loss function*. Μάλιστα η μέθοδος *cross-entropy* εμφανίζεται να έχει αισθητά καλύτερη απώλεια σε σχέση με την *exponential* μέθοδο. Βέβαια, αυτό έχει να κάνει και από το πώς ορίζουμε τα *loss-functions*.

Συγκρίνοντας τώρα τις 2 μεθόδους με το βέλτιστο τεστ κατά *Bayes*, λαμβάνουμε τα εξής αποτελέσματα :

Να επισημάνουμε εδώ ότι όπως φαίνεται και από το παραπάνω γράφημα, ο *Bayes* συνιστά το βέλτιστο τεστ απόφασης, πράγμα το οποίο αντιστοιχεί με την θεωρία. Ωστόσο, παρατηρούμε ότι και οι 3 μέθοδοι παρουσιάζουν πιθανότητες σφάλματος οι οποίες είναι εξαιρετικά κοντά μεταξύ τους, γεγονός το οποίο δικαιολογείται από το σχετικά μεγάλο αριθμό δειγμάτων (10^6).



Εικόνα 3: Σύγκριση της συνολικής πιθανότητας λάθους εφαρμόζοντας τον *Bayes* (σύμφωνα με την θεωρία είναι η βέλτιστη) τη μέθοδο *cross-entropy* και τη μέθοδο *exponential*

Πρόβλημα 2

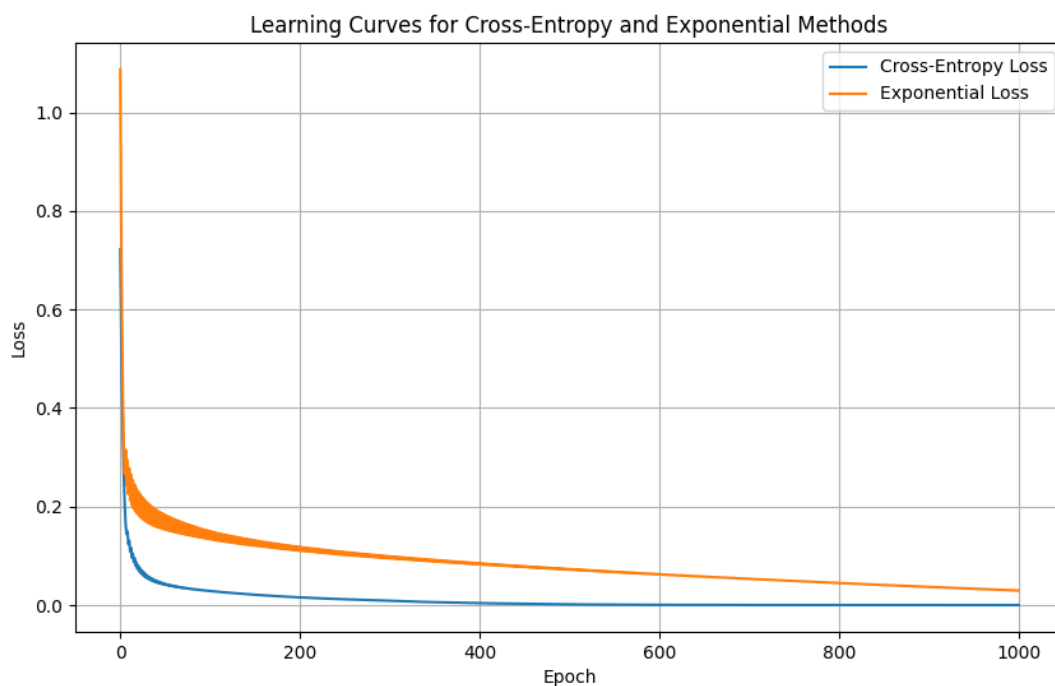
Στο 2^ο πρόβλημα αυτό που κάνουμε είναι να εφαρμόσουμε την ιδέα της προηγούμενης άσκησης. Συγκεκριμένα, καλούμαστε να δημιουργήσουμε 2 νευρωνικά δίκτυα (εκ των οποίων το 1 θα κάνει χρήση της *cross-entropy method* και το άλλο της *exponential*) προκειμένου να αναγνωριστούν τα ψηφία 0 και 8. Γι' αυτό το λόγο χρησιμοποιούμε τη βάση δεδομένων **MNIST** όπου κάνουμε *isolate* τα ψηφία 0 και 8. Έτσι λοιπόν η βασική δομή του νευρωνικού δικτύου περιλαμβάνει 784 εισόδους (1 εικόνα = 28×28), 300 νευρώνες στο *hidden layer* και μία έξοδο η οποία προβλέπει αν η αρχική είσοδος αντιστοιχεί σε 0/8. Για κάθε *pixel*, το κανονικοποιούμε έτσι ώστε η τιμή του να βρίσκεται στο $[0,1]$. Δημιουργώντας λοιπόν τα σχετικά νευρωνικά δίκτυα με τη λογική των προηγούμενων ερωτημάτων και αφού πρώτα τα κάνουμε *train* σε ένα μικρό αριθμό δεδομένων βλέπουμε την απόδοσή τους. Όπως άλλωστε επιβεβαιώνεται εν τέλει αναμένουμε να έχουμε (εξαιρετικά) χαμηλά ποσοστά λάθους μιας και τα ψηφία 0 και 8

αποδεικνύονται εύκολα στην διάκρισή τους. Τρέχοντας λοιπόν τον σχετικό κώδικα, λαμβάνουμε τα εξής αποτελέσματα (Σημείωση: ο σχετικός κώδικας απαιτεί αρκετό χρόνο για να τρέξει όπως επίσης και μεγάλη υπολογιστική ικανότητα για τον υπολογιστή ο οποίος θα τον τρέξει).

Προκύπτει ο παρακάτω πίνακας:

<i>Numeral/Error rate</i>	<i>Cross-Entropy method</i>	<i>Exponential method</i>
0	0.41 %	0.41 %
8	0.72 %	0.51 %
Total error rate	0.56 %	0.46 %

Και τα γραφήματα τα οποία δείχνουν τη σύγκλιση των 2 μεθόδων:



Εικόνα 4: Learning curves για το νευρωνικό δίκτυο το οποίο διαχωρίζει στην έξοδό του το ψηφίο 0 από το ψηφίο 8 (2 εκδοχές του)

Σύμφωνα με τα παραπάνω η *cross-entropy method* εμφανίζει ταχύτερη σύγκλιση και μικρότερο σφάλμα εν συγκρίσει με την *exponential method*. Εντούτοις, ακριβώς επειδή ακριβώς ο αλγόριθμος SGD συνιστά έναν ασταθή αλγόριθμο υπάρχει περίπτωση ύστερα από κάποιο συγκεκριμένο αριθμό από epochs το loss να αυξάνει.

Παρακάτω παρατίθενται οι κώδικες για τα 2 προβλήματα (και οι 2 δημιουργήθηκαν με τη βοήθεια του γνωστού εργαλείου Chat-GPT αλλά με συνεχόμενο prompting και έλεγχο από τον συντάκτη της παρούσας εργασίας). Έχουν πολλές γραμμές μιας και δε χρησιμοποιήθηκαν έτοιμα πακέτα ενώ μόλις έγινε γνωστό ότι επιτρεπόταν η χρήση έτοιμων πακέτων, τα χρονικά περιθώρια ήταν ήδη πολύ στενά. Επίσης, για να τρέξουν οι κώδικες απαιτούνται τα εξής (στο cmd των Windows)

1. *pip install numpy*
2. *pip install matplotlib*
3. Στον κώδικα για το 2^ο πρόβλημα απαιτείται προσαρμογή του *path* στο οποίο βρίσκονται τα *MNIST data*

```

import numpy as np
import matplotlib.pyplot as plt

# -----
# Step 1: Generate Data
# -----

# Number of samples
num_samples = 10**6

# Generate H0 data:  $N(0,1)$  for both  $x_1$  and  $x_2$ 
f0_data = np.random.normal(0, 1, (num_samples, 2))

# Generate H1 data: mixture of  $N(1,1)$  and  $N(-1,1)$  for both  $x_1$  and  $x_2$ 
mixture_choices = np.random.rand(num_samples, 2) > 0.5 # True for  $N(1,1)$ ,
False for  $N(-1,1)$ 
f1_data = np.where(
    mixture_choices,
    np.random.normal(1, 1, (num_samples, 2)),
    np.random.normal(-1, 1, (num_samples, 2))
)

# -----
# Step 2: Compute Bayes Optimal Test
# -----

def compute_log_likelihood_ratios(x):
    """
    Computes the log-likelihood ratio  $\log(r(x_1, x_2))$  where

```

```
r(x1, x2) = (f1(x1)/f0(x1)) * (f1(x2)/f0(x2))
"""
```

```
# Log-likelihoods under f0
```

```
log_f0 = -0.5 * x**2 - np.log(np.sqrt(2 * np.pi))
```

```
# Log-likelihoods under f1 (mixture)
```

```
log_f1_pos = -0.5 * (x - 1)**2 - np.log(np.sqrt(2 * np.pi))
```

```
log_f1_neg = -0.5 * (x + 1)**2 - np.log(np.sqrt(2 * np.pi))
```

```
# Compute log(f1) using log-sum-exp for numerical stability
```

```
max_log = np.maximum(log_f1_pos, log_f1_neg)
```

```
log_f1 = max_log + np.log(0.5 * np.exp(log_f1_pos - max_log) + 0.5 *  
np.exp(log_f1_neg - max_log))
```

```
# Log-likelihood ratios for each dimension
```

```
log_r = log_f1 - log_f0 # Shape: (num_samples, 2)
```

```
# Total log-likelihood ratio
```

```
total_log_r = np.sum(log_r, axis=1) # Shape: (num_samples,)
```

```
return total_log_r
```

```
# Compute log-likelihood ratios for H0 and H1 data
```

```
log_r_f0 = compute_log_likelihood_ratios(f0_data)
```

```
log_r_f1 = compute_log_likelihood_ratios(f1_data)
```

```
# Decisions based on log-likelihood ratios
```

```
decisions_f0 = (log_r_f0 > 0).astype(int) # H1 if log(r) > 0
```

```
decisions_f1 = (log_r_f1 > 0).astype(int) # H1 if log(r) > 0
```

```
# Error probabilities
```

```
error_f0 = np.mean(decisions_f0 != 0) # Error: H0 -> H1
```

```
error_f1 = np.mean(decisions_f1 != 1) # Error: H1 -> H0
total_error_probability = 0.5 * (error_f0 + error_f1)
```

```
print(f"Error probability for H0: {error_f0:.4f}")
print(f"Error probability for H1: {error_f1:.4f}")
print(f"Total probability of error (Bayes Optimal):
{total_error_probability:.4f}")
```

```
# Plot Bayes Optimal Errors
```

```
labels_plot = ['H0 Error', 'H1 Error', 'Total Error']
values = [error_f0, error_f1, total_error_probability]
```

```
plt.figure(figsize=(8, 6))
plt.bar(labels_plot, values, color=['blue', 'orange', 'green'])
plt.xlabel('Error Type')
plt.ylabel('Probability')
plt.title('Bayes Optimal Error Probabilities')
plt.ylim(0, 1)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

```
# -----
```

```
# Step 3: Prepare Training Data
```

```
# -----
```

```
train_samples = 200 # Number of training samples per hypothesis
```

```
# Training data for H0: N(0,1)
```

```
train_f0_data = np.random.normal(0, 1, (train_samples, 2))
```

```
# Training data for H1: mixture of N(1,1) and N(-1,1)
mixture_choices_train = np.random.rand(train_samples, 2) > 0.5
train_f1_data = np.where(
    mixture_choices_train,
    np.random.normal(1, 1, (train_samples, 2)),
```

```
    np.random.normal(-1, 1, (train_samples, 2))
)
```

```
# Combine training data and labels
```

```
train_data = np.vstack((train_f0_data, train_f1_data)) # Shape: (400, 2)
train_labels = np.hstack((np.zeros(train_samples),
    np.ones(train_samples))).reshape(-1, 1) # Shape: (400, 1)
```

```
# Compute log-likelihood ratios for training data
```

```
log_r_train = compute_log_likelihood_ratios(train_data)
```

```
# Targets for Cross-Entropy Method:  $P(H1|x) = r / (1 + r)$ 
```

```
ce_targets = (np.exp(log_r_train) / (1 + np.exp(log_r_train))).reshape(-1, 1)
# Shape: (400, 1)
```

```
# Targets for Exponential Method: transform labels to -1 or +1
```

```
exp_targets = 2 * train_labels - 1 # Transforms labels from {0,1} to {-1,+1}
```

```
# -----
```

```
# Step 4: Define Neural Network Components
```

```
# -----
```

```
def relu(x):
    return np.maximum(0, x)
```

```
def relu_derivative(x):
```

```
return (x > 0).astype(float)
```

```
def sigmoid(x):
```

```
    return 1 / (1 + np.exp(-x))
```

```
def initialize_parameters(input_size, hidden_size, output_size):
```

```
    """
```

initializes weights and biases.

Weights are drawn from $N(0, 1/(n + m))$ where n and m are input and output dimensions.

Biases are initialized to zero.

```
    """
```

```
    weights_1 = np.random.normal(0, np.sqrt(1 / (input_size + hidden_size)),  
                                  (input_size, hidden_size))
```

```
    biases_1 = np.zeros(hidden_size)
```

```
    weights_2 = np.random.normal(0, np.sqrt(1 / (hidden_size +  
output_size)), (hidden_size, output_size))
```

```
    biases_2 = np.zeros(output_size)
```

```
    return weights_1, biases_1, weights_2, biases_2
```

```
def custom_cross_entropy_loss(Y, targets):
```

```
    """
```

Computes the binary cross-entropy loss.

Args:

Y: Predicted probabilities (N, 1).

targets: True probabilities (N, 1).

Returns:

Scalar loss value.

```
    """
```

```
    epsilon = 1e-8 # To prevent log(0)
```

```
    return np.mean(-targets * np.log(Y + epsilon) - (1 - targets) * np.log(1 - Y +  
epsilon))
```

```
def custom_exponential_loss(Y, targets):
```

```
    """
```

Computes the standard exponential loss.

Args:

Y: Predicted raw outputs (N, 1).

targets: Transformed labels (-1 or +1) (N, 1).

Returns:

Scalar loss value.

"""

```
return np.mean(np.exp(- targets * Y))
```

```
def sgd_with_adam_normalization(param, grad, P_prev, mu=0.001,  
lambda_=0.999, c=1e-8):
```

"""

Performs the SGD update with ADAM-like normalization on a parameter.

Args:

param: Current parameter value.

grad: Gradient of the loss w.r.t. the parameter.

P_prev: Previous power estimate (P[t-1]).

mu: Learning rate.

lambda_: Decay rate for the power estimate.

c: Small constant to prevent division by zero.

Returns:

Updated parameter, updated P.

"""

Update power estimate

```
P = (1 - lambda_) * P_prev + lambda_ * (grad ** 2)
```

Update parameter with normalized gradient

```
param_update = mu * grad / (np.sqrt(c + P))
```

```
param -= param_update
```

```
return param, P
```

```
def initialize_power_estimates(shape):
```

"""

Initializes the power estimates P for a given parameter shape.

Starts with zeros.

"""

```
return np.zeros(shape)
```

```
def forward_pass(data, weights_1, biases_1, weights_2, biases_2,  
activation_output='sigmoid'):
```

"""

Performs a forward pass through the network.

Args:

data: Input data of shape (N, 2).

weights_1: Weights of the first layer (2, 20).

biases_1: Biases of the first layer (20,).

weights_2: Weights of the second layer (20, 1).

biases_2: Biases of the second layer (1,).

activation_output: 'sigmoid' or 'none'.

Returns:

Y: Output activations (N, 1).

Z1: Hidden layer activations (N, 20).

"""

```
W1 = np.dot(data, weights_1) + biases_1 # Shape: (N, 20)
```

```
Z1 = relu(W1) # Shape: (N, 20)
```

```
W2 = np.dot(Z1, weights_2) + biases_2 # Shape: (N, 1)
```

```
if activation_output == 'sigmoid':
```

```
    Y = sigmoid(W2) # Shape: (N, 1)
```

```
elif activation_output == 'none':
```

```
    Y = W2 # Shape: (N, 1)
```

```
else:
```

```
    raise ValueError("Invalid activation_output")
```

```
return Y, Z1
```

```
def compute_gradients_custom(X, Z1, Y, targets, weights_1, biases_1,
weights_2, method='cross_entropy'):
```

```
    """
```

Computes the gradients based on the provided loss functions.

Args:

X: Input data (N, 2).

Z1: Hidden layer activations (N, 20).

Y: Output activations (N, 1).

targets: True labels or targets (N, 1).

weights_1: Weights of the first layer (2, 20).

biases_1: Biases of the first layer (20,).

weights_2: Weights of the second layer (20, 1).

method: 'cross_entropy' or 'exponential'.

Returns:

Gradients for weights_2, biases_2, weights_1, biases_1.

```
    """
```

```
    N = X.shape[0]
```

```
    if method == 'cross_entropy':
```

```
        # For cross-entropy loss with sigmoid activation
```

```
        U2 = Y - targets # Shape: (N, 1)
```

```
        f2_prime = Y * (1 - Y) # Derivative of sigmoid
```

```
    elif method == 'exponential':
```

```
        # For exponential loss with linear activation
```

```
        U2 = - targets * np.exp(- targets * Y) # Shape: (N, 1)
```

```
        f2_prime = np.ones_like(Y) # Derivative of linear activation is 1
```

```
    else:
```

```
        raise ValueError("Invalid method")
```

```
    V2 = U2 * f2_prime # Shape: (N, 1)
```

```
    # Compute gradients for weights_2 and biases_2
```

```
    grad_weights_2 = np.dot(Z1.T, V2) / N # Shape: (20, 1)
```

```
    grad_biases_2 = np.sum(V2, axis=0) / N # Shape: (1,)
```

```

# Compute  $V1 = (V2 \text{ dot } weights\_2^T) \odot f'_1(W1)$ 
f1_prime = relu_derivative(np.dot(X, weights_1) + biases_1) # Shape: (N, 20)
V1 = np.dot(V2, weights_2.T) * f1_prime # Shape: (N, 20)

```

```

# Compute gradients for weights_1 and biases_1
grad_weights_1 = np.dot(X.T, V1) / N # Shape: (2, 20)
grad_biases_1 = np.sum(V1, axis=0) / N # Shape: (20,)

return grad_weights_2, grad_biases_2, grad_weights_1, grad_biases_1

```

```

# -----

```

```

# Step 5: Training Function with SGD and ADAM-like Normalization

```

```

# -----

```

```

def train_network_sgd_adam_norm(train_data, targets,
method='cross_entropy', num_epochs=500, mu=0.001, lambda_=0.999,
c=1e-8, hidden_size=20):
    """

```

Trains a neural network using SGD with ADAM-like normalization.

Args:

train_data: Training data of shape (400, 2).

targets: Targets for training of shape (400, 1).

method: 'cross_entropy' or 'exponential'.

num_epochs: Number of training epochs.

mu: Learning rate.

lambda_: Decay rate for the power estimate.

c: Small constant to prevent division by zero.

hidden_size: Number of neurons in the hidden layer.

Returns:

Trained weights and biases, and history of loss values.

```

"""
input_size = 2
output_size = 1

# Initialize parameters
weights_1, biases_1, weights_2, biases_2 =
initialize_parameters(input_size, hidden_size, output_size)
weights_1, biases_1, weights_2, biases_2 = initialize_parameters(input_size,
hidden_size, output_size)
# Initialize power estimates for each parameter matrix
P_w1 = initialize_power_estimates(weights_1.shape)
P_b1 = initialize_power_estimates(biases_1.shape)
P_w2 = initialize_power_estimates(weights_2.shape)
P_b2 = initialize_power_estimates(biases_2.shape)

history = []

for epoch in range(1, num_epochs + 1):
    # Forward pass
    Y, Z1 = forward_pass(train_data, weights_1, biases_1, weights_2,
biases_2,
                        activation_output='sigmoid' if method == 'cross_entropy'
else 'none')

    # Compute loss
    if method == 'cross_entropy':
        loss = custom_cross_entropy_loss(Y, targets)
    elif method == 'exponential':
        loss = custom_exponential_loss(Y, targets)
    else:
        raise ValueError("Invalid training method")

```

```

history.append(loss)

# Compute gradients
grad_weights_2, grad_biases_2, grad_weights_1, grad_biases_1 =
compute_gradients_custom(
    train_data, Z1, Y, targets, weights_1, biases_1, weights_2,
    method=method)

# Update weights and biases using SGD with ADAM-like normalization
weights_2, P_w2 = sgd_with_adam_normalization(weights_2,
grad_weights_2, P_w2, mu, lambda_, c)
biases_2, P_b2 = sgd_with_adam_normalization(biases_2,
grad_biases_2, P_b2, mu, lambda_, c)
weights_1, P_w1 = sgd_with_adam_normalization(weights_1,
grad_weights_1, P_w1, mu, lambda_, c)
biases_1, P_b1 = sgd_with_adam_normalization(biases_1,
grad_biases_1, P_b1, mu, lambda_, c)

# Print smoothed cost every 50 epochs
if epoch % 50 == 0:
    # Smoothed loss: average of the last 20 epochs
    smoothed_loss = np.mean(history[-20:])
    print(f"Epoch {epoch}, Smoothed Cost ({method.capitalize()} SGD
with ADAM-like Norm): {smoothed_loss:.6f}")

return weights_1, biases_1, weights_2, biases_2, history

```

```

# -----
# Step 6: Train Both Neural Networks
# -----

# Train Cross-Entropy Network
print("\nTraining Cross-Entropy Network with SGD and ADAM-like
Normalization:")
weights_1_ce, biases_1_ce, weights_2_ce, biases_2_ce, history_ce =
train_network_sgd_adam_norm(
    train_data, ce_targets, method='cross_entropy', num_epochs=500,
    mu=0.001, lambda_=0.999, c=1e-8, hidden_size=20)
# Train Exponential Network
print("\nTraining Exponential Network with SGD and ADAM-like
Normalization:")
weights_1_exp, biases_1_exp, weights_2_exp, biases_2_exp, history_exp =
train_network_sgd_adam_norm(
    train_data, exp_targets, method='exponential', num_epochs=500,
    mu=0.001, lambda_=0.999, c=1e-8, hidden_size=20)

# -----
# Step 7: Plot Training Loss
# -----

plt.figure(figsize=(10, 6))
plt.plot(history_ce, label='Cross-Entropy Loss with SGD + ADAM-like Norm')
plt.plot(history_exp, label='Exponential Loss with SGD + ADAM-like Norm',
color='orange')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss over Epochs for Cross-Entropy and Exponential
Methods')
plt.legend()
plt.grid(True)
plt.show()

```

```

# -----
# Step 8: Evaluate Neural Networks on Test Data
# -----

def compute_decisions(Y, method='cross_entropy'):
    """
    Converts network outputs to binary decisions based on the method.
    Args:
        Y: Network outputs.
        method: 'cross_entropy' or 'exponential'.
    Returns:
        Binary decisions: 0 or 1.
    """
    if method == 'cross_entropy':
        # Decide H1 if  $P(H1|x) > 0.5$ 
        decisions = (Y.flatten() > 0.5).astype(int)
    elif method == 'exponential':
        # Decide H1 if  $Y > 0$  (since targets are -1 or +1)
        decisions = (Y.flatten() > 0).astype(int)
    else:
        raise ValueError("Invalid method for decision computation")
    return decisions

def evaluate_network_in_batches(test_data, weights_1, biases_1,
                               weights_2, biases_2, method='cross_entropy', batch_size=100000):
    """
    Evaluates the network on test data in batches to manage memory usage.
    Args:
        test_data: Test data of shape (2,000,000, 2).
    """

```

weights_1: Weights of the first layer.
biases_1: Biases of the first layer.
weights_2: Weights of the second layer.

```
biases_2: Biases of the second layer.  
method: 'cross_entropy' or 'exponential'.  
batch_size: Number of samples per batch.  
Returns:  
    decisions: Binary decisions for all test samples.  
""""  
  
num_test = test_data.shape[0]  
decisions = np.zeros(num_test, dtype=int)  
  
for start in range(0, num_test, batch_size):  
    end = min(start + batch_size, num_test)  
    batch = test_data[start:end]  
    Y, _ = forward_pass(batch, weights_1, biases_1, weights_2, biases_2,  
                          activation_output='sigmoid' if method == 'cross_entropy'  
else 'none')  
    batch_decisions = compute_decisions(Y, method=method)  
    decisions[start:end] = batch_decisions  
  
return decisions  
  
# Prepare Test Data  
test_data = np.vstack((f0_data, f1_data)) # Shape: (2,000,000, 2)  
test_labels = np.hstack((np.zeros(num_samples), np.ones(num_samples))) #  
Shape: (2,000,000,)  
  
# Evaluate Cross-Entropy Network on Test Data  
print("\nEvaluating Cross-Entropy Network on Test Data:")
```

```

decisions_ce = evaluate_network_in_batches(test_data, weights_1_ce,
biases_1_ce, weights_2_ce, biases_2_ce, method='cross_entropy')

# Evaluate Exponential Network on Test Data
print("\nEvaluating Exponential Network on Test Data:")
decisions_exp = evaluate_network_in_batches(test_data, weights_1_exp,
biases_1_exp, weights_2_exp, biases_2_exp, method='exponential')

# Compute error probabilities for Cross-Entropy method
error_ce_f0 = np.mean(decisions_ce[test_labels == 0] != 0)
error_ce_f1 = np.mean(decisions_ce[test_labels == 1] != 1)
total_error_ce = 0.5 * (error_ce_f0 + error_ce_f1)

# Compute error probabilities for Exponential method
error_exp_f0 = np.mean(decisions_exp[test_labels == 0] != 0)
error_exp_f1 = np.mean(decisions_exp[test_labels == 1] != 1)
total_error_exp = 0.5 * (error_exp_f0 + error_exp_f1)

# Print probabilities for all methods
print(f"\nBayes Optimal Total Error Probability:
{total_error_probability:.6f}")
print(f"Cross-Entropy Method Total Error Probability: {total_error_ce:.6f}")
print(f"Exponential Method Total Error Probability: {total_error_exp:.6f}")

# Plot comparison of error probabilities
labels_compare = ['Bayes Optimal', 'Cross-Entropy', 'Exponential']
values_compare = [total_error_probability, total_error_ce, total_error_exp]

plt.figure(figsize=(8, 6))
plt.bar(labels_compare, values_compare, color=['green', 'blue', 'orange'])
plt.xlabel('Method')
plt.ylabel('Total Error Probability')
plt.title('Comparison of Total Error Probabilities')
plt.ylim(0, max(values_compare) + 0.05)
for i, v in enumerate(values_compare):

```

```
plt.text(i, v + 0.005, f"{v:.6f}", ha='center')  
plt.grid(axis='y', linestyle='--', alpha=0.7)  
plt.show()
```

Και για το πρόβλημα 2:

```
import os
import numpy as np
import matplotlib.pyplot as plt

# -----
# 1. Data Preparation
# -----

def load_mnist_images(filename):
    with open(filename, 'rb') as f:
        f.read(4) # Magic number
        num_images = int.from_bytes(f.read(4), 'big')
        num_rows = int.from_bytes(f.read(4), 'big')
        num_cols = int.from_bytes(f.read(4), 'big')
        buffer = f.read(num_images * num_rows * num_cols)
        data = np.frombuffer(buffer, dtype=np.uint8).reshape(num_images,
num_rows * num_cols)
        return data / 255.0 # Normalize to [0,1]

def load_mnist_labels(filename):
    with open(filename, 'rb') as f:
```

```
f.read(4) # Magic number
num_labels = int.from_bytes(f.read(4), 'big')
buffer = f.read(num_labels)
return np.frombuffer(buffer, dtype=np.uint8)
```

```
def prepare_data():
    # Set the folder path to your mnist_data directory
    folder = r"C:\Users\georg\Desktop\deep_learning_hw\mnist_data"

    # Full paths for each file
```

```
train_images_path = os.path.join(folder, 'train-images.idx3-ubyte')
train_labels_path = os.path.join(folder, 'train-labels.idx1-ubyte')
test_images_path = os.path.join(folder, 't10k-images.idx3-ubyte')
test_labels_path = os.path.join(folder, 't10k-labels.idx1-ubyte')
```

```
# Print each path to confirm
```

```
print(f"Train images path: {train_images_path}")
print(f"Train labels path: {train_labels_path}")
print(f"Test images path: {test_images_path}")
print(f"Test labels path: {test_labels_path}")
```

```
# Check if files exist
```

```
assert os.path.exists(train_images_path), f"File not found:
{train_images_path}"
assert os.path.exists(train_labels_path), f"File not found:
{train_labels_path}"
assert os.path.exists(test_images_path), f"File not found:
{test_images_path}"
assert os.path.exists(test_labels_path), f"File not found:
{test_labels_path}"
```

Load the data

```
train_images = load_mnist_images(train_images_path)
train_labels = load_mnist_labels(train_labels_path)
test_images = load_mnist_images(test_images_path)
test_labels = load_mnist_labels(test_labels_path)
```

Filter for digits 0 and 8

```
train_filter = np.where((train_labels == 0) | (train_labels == 8))
test_filter = np.where((test_labels == 0) | (test_labels == 8))
X_train, y_train = train_images[train_filter], train_labels[train_filter]
X_test, y_test = test_images[test_filter], test_labels[test_filter]
```

Map labels: 0 stays 0, 8 becomes 1

```
y_train = np.where(y_train == 8, 1, 0)
y_test = np.where(y_test == 8, 1, 0)
```

Verify class distribution

```
print('Training samples per class:', np.unique(y_train,
return_counts=True))
print('Testing samples per class:', np.unique(y_test, return_counts=True))
```

```
return X_train, y_train, X_test, y_test
```

-----

2. Neural Network with Specified Initialization

-----

```
def initialize_parameters(input_size, hidden_size, output_size):
    weights_1 = np.random.normal(0, np.sqrt(1 / (input_size + hidden_size)),
(input_size, hidden_size))
    biases_1 = np.zeros((1, hidden_size))
    weights_2 = np.random.normal(0, np.sqrt(1 / (hidden_size +
output_size)), (hidden_size, output_size))
    biases_2 = np.zeros((1, output_size))
```

```
return weights_1, biases_1, weights_2, biases_2
```

```
def relu(x):  
    return np.maximum(0, x)
```

```
def sigmoid(x):  
    return 1 / (1 + np.exp(-x))
```

```
# -----
```

```
# 3. Forward and Backward Passes with Different Loss Functions
```

```
# -----
```

```
def forward_pass(X, weights_1, biases_1, weights_2, biases_2,  
method='cross_entropy'):
```

```
    Z1 = np.dot(X, weights_1) + biases_1
```

```
    A1 = relu(Z1)
```

```
    Z2 = np.dot(A1, weights_2) + biases_2
```

```
    if method == 'cross_entropy':
```

```
        A2 = sigmoid(Z2) # Output as probability
```

```
    elif method == 'exponential':
```

```
        A2 = Z2 # Output as log-likelihood ratio directly
```

```
    else:
```

```
        raise ValueError("Invalid method specified.")
```

```
    return {'X': X, 'Z1': Z1, 'A1': A1, 'Z2': Z2, 'A2': A2}
```

```
def compute_loss(A2, Y, method='cross_entropy'):
```

```
    m = Y.shape[0]
```

```
    if method == 'cross_entropy':
```

```
        epsilon = 1e-8
```

```
        A2 = np.clip(A2, epsilon, 1 - epsilon)
```

```
        loss = - (1 / m) * np.sum(Y * np.log(A2) + (1 - Y) * np.log(1 - A2))
```

```
    elif method == 'exponential':
```

```
        Y_mapped = 2 * Y - 1 # Map 0 -> -1, 1 -> 1
```

```

        loss = (1 / m) * np.sum(np.exp(-Y_mapped * A2))
    else:
        raise ValueError("Invalid loss method specified.")
    return loss

def backward_pass(cache, Y, weights_2, method='cross_entropy'):
    m = Y.shape[0]
    A2 = cache['A2']
    A1 = cache['A1']

```

```

Z1 = cache['Z1']
X = cache['X']

if method == 'cross_entropy':
    dZ2 = A2 - Y
elif method == 'exponential':
    Y_mapped = 2 * Y - 1 # Map 0 -> -1, 1 -> 1
    dZ2 = -Y_mapped * np.exp(-Y_mapped * A2)
else:
    raise ValueError("Invalid method specified.")

dW2 = np.dot(A1.T, dZ2) / m
db2 = np.sum(dZ2, axis=0, keepdims=True) / m
dA1 = np.dot(dZ2, weights_2.T)
dZ1 = dA1 * (Z1 > 0).astype(float)
dW1 = np.dot(X.T, dZ1) / m
db1 = np.sum(dZ1, axis=0, keepdims=True) / m

return {'dW1': dW1, 'db1': db1, 'dW2': dW2, 'db2': db2}

```

4. ADAM Optimizer for Parameter Updates

```
# -----
def adam_update(param, grad, P_prev, mu=0.001, lambda_=0.999, c=1e-8):
    P = (1 - lambda_) * P_prev + lambda_ * (grad ** 2)
    param_update = mu * grad / (np.sqrt(c + P))
    param -= param_update
    return param, P
```

```
# -----
# 5. Training Loop without Plotting (Plotting will be done in main script)
# -----

def train_network_adam(X_train, y_train, num_epochs=1000,
learning_rate=0.001, lambda_=0.999, method='cross_entropy'):
    input_size = X_train.shape[1]
    hidden_size = 300
    output_size = 1
    weights_1, biases_1, weights_2, biases_2 =
initialize_parameters(input_size, hidden_size, output_size)
    P_w1, P_b1 = np.zeros_like(weights_1), np.zeros_like(biases_1)
    P_w2, P_b2 = np.zeros_like(weights_2), np.zeros_like(biases_2)
    losses = []

    for epoch in range(num_epochs):
        cache = forward_pass(X_train, weights_1, biases_1, weights_2,
biases_2, method=method)
        loss = compute_loss(cache['A2'], y_train.reshape(-1, 1),
method=method)
        losses.append(loss)
```

```

    gradients = backward_pass(cache, y_train.reshape(-1, 1), weights_2,
method=method)

    weights_1, P_w1 = adam_update(weights_1, gradients['dW1'], P_w1,
mu=learning_rate, lambda_=lambda_)
    biases_1, P_b1 = adam_update(biases_1, gradients['db1'], P_b1,
mu=learning_rate, lambda_=lambda_)
    weights_2, P_w2 = adam_update(weights_2, gradients['dW2'], P_w2,
mu=learning_rate, lambda_=lambda_)
    biases_2, P_b2 = adam_update(biases_2, gradients['db2'], P_b2,
mu=learning_rate, lambda_=lambda_)

    if epoch % 100 == 0:
        print(f"Epoch {epoch}, Loss: {loss}")

    return weights_1, biases_1, weights_2, biases_2, losses

```

```

# -----
# 6. Evaluation
# -----

```

```

def evaluate(X, y, weights_1, biases_1, weights_2, biases_2,
method='cross_entropy'):
    cache = forward_pass(X, weights_1, biases_1, weights_2, biases_2,
method=method)
    if method == 'cross_entropy':
        predictions = (cache['A2'] > 0.5).astype(int)
    elif method == 'exponential':
        predictions = (cache['A2'] > 0).astype(int)
    error_rate = np.mean(predictions != y.reshape(-1, 1))
    return error_rate, predictions.flatten()

```

```

# -----
# Main Script
# -----

```

```

if __name__ == "__main__":
    X_train, y_train, X_test, y_test = prepare_data()

    # Train and collect losses for Cross-Entropy Method
    print("\nTraining with Cross-Entropy Method:")
    w1_ce, b1_ce, w2_ce, b2_ce, losses_ce = train_network_adam(X_train,
y_train, method='cross_entropy')

    # Train and collect losses for Exponential Method
    print("\nTraining with Exponential Method:")
    w1_exp, b1_exp, w2_exp, b2_exp, losses_exp =
train_network_adam(X_train, y_train, method='exponential')

    # Plot learning curves for both methods on the same graph
    plt.figure(figsize=(10, 6))
    plt.plot(losses_ce, label='Cross-Entropy Loss')
    plt.plot(losses_exp, label='Exponential Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Learning Curves for Cross-Entropy and Exponential Methods')
    plt.legend()
    plt.grid(True)
    plt.show()

    # Evaluate on test set for both methods
    error_rate_ce, predictions_ce = evaluate(X_test, y_test, w1_ce, b1_ce,
w2_ce, b2_ce, method='cross_entropy')
    error_rate_exp, predictions_exp = evaluate(X_test, y_test, w1_exp,
b1_exp, w2_exp, b2_exp, method='exponential')

    print(f"Cross-Entropy Test Error Rate: {error_rate_ce * 100:.2f}%")
    print(f"Exponential Test Error Rate: {error_rate_exp * 100:.2f}%")

    # Create a table of error rates per numeral
    def error_per_class(y_true, predictions):

```

```
classes = np.unique(y_true)
error_rates = {}
for cls in classes:
    idx = np.where(y_true == cls)
    error = np.mean(predictions[idx] != y_true[idx])
    error_rates[cls] = error * 100
return error_rates
```

Map labels back to numerals

```
label_to_numeral = {0: 0, 1: 8}
```

Error rates per numeral for Cross-Entropy Method

```
error_rates_ce = error_per_class(y_test, predictions_ce)
```

```
print("\nError Rates per Numeral for Cross-Entropy Method:")
```

```
for label, error_rate in error_rates_ce.items():
```

```
    numeral = label_to_numeral[label]
```

```
    print(f"Numeral {numeral}: {error_rate:.2f}%")
```

```
print(f"Total Error Rate: {error_rate_ce * 100:.2f}%")
```

Error rates per numeral for Exponential Method

```
error_rates_exp = error_per_class(y_test, predictions_exp)
```

```
print("\nError Rates per Numeral for Exponential Method:")
```

```
for label, error_rate in error_rates_exp.items():
```

```
    numeral = label_to_numeral[label]
```

```
    print(f"Numeral {numeral}: {error_rate:.2f}%")
```

```
print(f"Total Error Rate: {error_rate_exp * 100:.2f}%")
```